

Assignment 7: Network traffic monitoring using the Snort intrusion detection system

Author: Δημήτρης Σταθούλιας

AM: 2018030109

Date: 2/1/2026

Part 2: Creating Custom Network Traffic

In this part of the assignment, we generated several distinct network packets simulating traffic in order to test the snort ruleset we created in part 3 below. The Python script and generated traffic.pcap file are included in the assignment directory including comments explaining design choices.

Part 3: Creating Custom Snort Rules

For this part of the assignment, we created a custom snort ruleset that raises alerts when detecting packets with specific characteristics. This ruleset involves several unique scenarios we generated with the prior python script in part 2:

- **Student packet:** This rule checks the packets of the generated traffic.pcap file for a payload containing the string “Dimitris Stathoulas-2018030109”, using a TCP protocol and having destination IP: 192.168.1.1 and port 54321.
- **Service ports packets:** This rule is written to identify packets sent to the IP: 192.168.1.2 with a payload containing the prior string “Dimitris Stathoulas-2018030109”. The rule is intentionally broad to serve a catch-all purpose of identifying all the possible service port packets matching the above specifications. Alternatively, we could instead implement an individual rule

for each service to make them more personalized. An example for the HTTP service is:

alert tcp any any -> 192.168.1.2 80 \

(content: "Dimitris Stathoulas-2018030109"; sid: 1000002; rev:1;)

- **Base64 packet:** This rule identifies packets sent to the IP:192.168.1.3 and port number 8080 using the TCP protocol and containing the base64 encoded AM:2018030109 -> "MjAxODAzMDEwOQ==".
- **Suspicious DNS packet:** This rule identifies packets containing a malicious domain name: malicious.example.com made using the DNS service, in other words using the UDP protocol and port number 53. The destination IP address used was 192.168.178.1.
- **Ping test Packet:** This rule identifies a packet made using the ICMP protocol to the IP:192.168.1.4 and containing the string "PingTest2025".

Below is presented a screenshot showing the alerts snort created when analyzing the generated traffic.pcap file using our custom local.rules ruleset.

```
++ [0] traffic.pcap
01/02-13:45:32.084018 [**] [1:1000001:1] "DETECT Student Packet" [**] [Priority: 0] {TCP} 178.24.157.223:20 -> 192.168.1.1:54321
01/02-13:45:32.085165 [**] [1:1000002:1] "DETECT Port Packet" [**] [Priority: 0] {TCP} 102.207.128.206:20 -> 192.168.1.2:80
01/02-13:45:32.086000 [**] [1:1000002:1] "DETECT Port Packet" [**] [Priority: 0] {TCP} 148.167.101.191:20 -> 192.168.1.2:443
01/02-13:45:32.086533 [**] [1:1000002:1] "DETECT Port Packet" [**] [Priority: 0] {TCP} 155.80.255.56:20 -> 192.168.1.2:22
01/02-13:45:32.086771 [**] [1:1000002:1] "DETECT Port Packet" [**] [Priority: 0] {TCP} 10.235.252.168:20 -> 192.168.1.2:23
01/02-13:45:32.086938 [**] [1:1000002:1] "DETECT Port Packet" [**] [Priority: 0] {TCP} 57.82.160.178:20 -> 192.168.1.2:21
01/02-13:45:32.087088 [**] [1:1000002:1] "DETECT Port Packet" [**] [Priority: 0] {UDP} 170.190.148.209:53 -> 192.168.1.2:53
01/02-13:45:32.087250 [**] [1:1000002:1] "DETECT Port Packet" [**] [Priority: 0] {TCP} 124.120.155.178:20 -> 192.168.1.2:554
01/02-13:45:32.087396 [**] [1:1000002:1] "DETECT Port Packet" [**] [Priority: 0] {TCP} 223.249.230.138:20 -> 192.168.1.2:5432
01/02-13:45:32.088430 [**] [1:1000002:1] "DETECT Port Packet" [**] [Priority: 0] {TCP} 174.17.209.31:20 -> 192.168.1.2:3389
01/02-13:45:32.088995 [**] [1:1000002:1] "DETECT Port Packet" [**] [Priority: 0] {TCP} 5.33.163.21:20 -> 192.168.1.2:1883
01/02-13:45:32.089295 [**] [1:1000003:1] "DETECT Base64 Packet" [**] [Priority: 0] {TCP} 0.84.106.250:20 -> 192.168.1.3:8080
01/02-13:45:32.090354 [**] [1:1000004:1] "DETECT Suspicious DNS Packet Packet" [**] [Priority: 0] {UDP} 141.255.178.94:53 -> 192.168.178.1:53
01/02-13:45:32.090951 [**] [1:1000005:1] "DETECT Ping Packet" [**] [Priority: 0] {ICMP} 243.131.144.172 -> 192.168.1.4
-- [0] traffic.pcap
```

Part 4: Detect Slammer Worm using Snort

The slammer worm is a malware that targets a buffer overflow vulnerability in the Microsoft SQL server service. To do that, it injects the byte “0x04” into the packet payload followed by a long data string exploiting the vulnerability. The packet details as observed in Wireshark using the provided slammer.pcap file are as follows:

- **Protocol:** UDP
- **Source IP:** 213.76.212.22
- **Source port:** 20199
- **Destination IP:** 65.165.167.86
- **Destination port:** 1434
- **Payload:** starting with 0x04 byte.

Our detection strategy involves creating a custom snort rule that analyzes the pcap file and raises an alert when the above protocol, destination port and payload are detected in a packet. Below is presented the screenshot of the snort alert when analyzing the slammer.pcap file.

```
++ [0] slammer.pcap
10/10-22:02:49.239104 [**] [1:1000006:1] "DETECT Slammer Worm Packet" [**] [Priority: 0] {UDP} 213.76.212.22:20199 -> 65.165.167.86:1434
-- [0] slammer.pcap
```

Part 5: Evaluate Snort

In this part of the assignment, we used a large pcap (1.2GB) file to stress test the snort tool. The file used can be found in [this](#) (MACCDC 2011) url with file name maccdc2011_00009_20110312174207.pcap.gz (300.3MB).

When, analyzing this file using the snort tool the following metrics can be observed while also using the “top” command:

- **Execution time:** 188 seconds
- **CPU usage:** 100%

- **Memory usage:** 34.6%
- **Snort errors/warnings:** None

The snort tool looks at each packet in the pcap file individually and analyzes its contents for its protocol, source, destination IP addresses and ports and its payload which can be computationally intense. With the virtual environment using 3 CPU cores and 4GB of RAM, we can observe the maximum CPU load for the entire execution duration.

There are several ways to reduce the above computational load that involve both increasing the computational power of the virtual environment and reducing the snort execution load. Firstly, a major way to increase performance is by increasing the CPU cores allocated to the virtual environment since snort uses multithreading when executing its processes. Another way for achieving increased performance is by filtering trusted traffic from the pcap file before using the snort tool to analyze it. This way, we can significantly reduce the amount of packets snort has to look through.

Part 6: Theoretical Questions

1. Snort is an intrusion detection tool which captures real-time traffic and compares each packet against a database of predefined or custom rules. Snort analyzes each packet checking its headers and payload and if they match each condition defined in the rules, snort executes the appropriate action, usually sending an alert.
2. Signature-based detection has some major flaws. Firstly, this detection method uses a database to compare against real-world threats which is susceptible to unknown attacks. Additionally, these methods include recognizing malicious patterns which is easily bypassed by traffic encryption unless decryption is performed beforehand. Lastly, by slightly modifying the malware, an attacker can completely bypass detection.

3.

- **Pros:** Snort is a free open-sourced tool making it a valid cost-effective choice. It also supports creating custom rules which helps users personalize its use cases. Finally, while it has some innate flaws, it does cover a large number of attacks making it a reliable option.
- **Cons:** On the other hand, snort requires keeping its database and ruleset constantly up to date to function properly against potential threats. Additionally, its use in high traffic networks can cause snort to struggle to analyze every packet causing bottlenecks and potentially allowing attacks through.